

Internet of Things (IoT) Lab

Lab 02: Interrupts and Sensors

Georgios Mentzos, Miran Tobar, Jörg Henkel | May 21, 2026

CHAIR FOR EMBEDDED SYSTEMS (CES)

Pre-Requisites

- Finished all tasks from Lab01
- Ready for Lab01 assessment

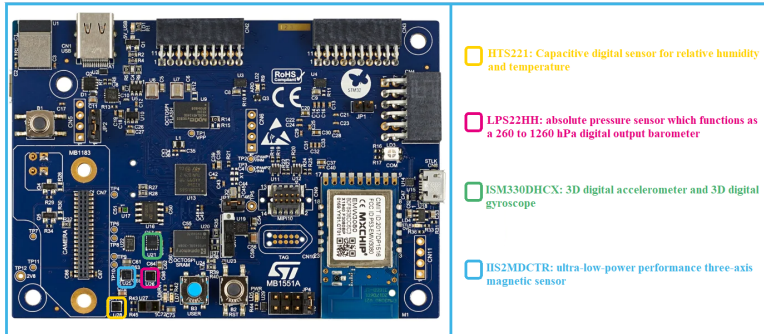
In this lab:

- Learn how to add the BSP for on-board sensors
- Learn how to use non-blocking UART
- Learn how to use interrupts and callbacks

Goal

By the end of this lab, you should be able to receive UART data without blocking the CPU and read values from on-board sensors using the BSP.

On-board sensors: Overview



- The board contains several environmental and motion sensors
- These sensors are connected through interfaces such as I²C
- In this lab, we access them through the BSP

For a good tutorial : https://wiki.st.com/stm32mcu/wiki/STM32StepByStep:Step4_Sensors_usage

[//wiki.st.com/stm32mcu/wiki/STM32StepByStep:Step4_Sensors_usage](https://wiki.st.com/stm32mcu/wiki/STM32StepByStep:Step4_Sensors_usage)

What is the BSP?

- BSP = Board Support Package
- It is a software layer between your application and the low-level component drivers
- It hides board-specific wiring details
- It provides easier-to-use functions such as:

```
BSP_ENV_SENSOR_Init(...)  
BSP_ENV_SENSOR_Enable(...)  
BSP_ENV_SENSOR_GetValue(...)
```

Abstraction

The component driver knows the sensor. The BSP knows how the sensor is connected on this specific board.

How to use the BSP

- CubeMX does not automatically add the BSP files
- The BSP comes from the STM32CubeU5 firmware package
- Copy the required files into:

```
Drivers/BSP/
```

- Add the required .c files to `target_sources(...)`
- Add the required include paths to `target_include_directories(...)`
- Create or copy the board configuration header:

```
b_u585i_iot02a_conf.h
```

Guide: <https://community.st.com/t5/stm32-mcus/how-to-add-a-bsp-to-an-stm32cubeide-project/ta-p/49812>. Adapt it to the B-U585I-IOT02A board and the STM32CubeU5 package.

- Digital relative humidity and temperature sensor
- Provides calibrated measurement values
- Supports I²C and SPI
- On this board, we access it via I²C through the BSP
- The BSP hides the raw calibration registers and exposes temperature and humidity values directly



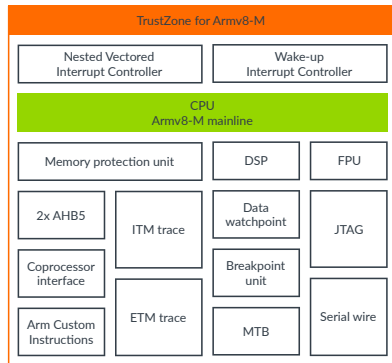
Figure: HTS221 sensor package

```
BSP_ENV_SENSOR_Init(0, ENV_TEMPERATURE);  
BSP_ENV_SENSOR_Enable(0, ENV_TEMPERATURE);  
BSP_ENV_SENSOR_GetValue(0, ENV_TEMPERATURE, &temperature);
```

Nested Vectored Interrupt Controller (NVIC)

- Part of the Arm Cortex-M33 core
- Supports up to 480 external interrupts
- Supports up to 256 priority levels
- Even with correct peripheral configuration, nothing happens if the interrupt is not enabled in the NVIC
- In this lab, we rarely access the NVIC directly. CubeMX and the HAL/CMSIS layer usually generate the required setup code.

arm CORTEX-M33



Source:

<https://developer.arm.com/-/media/arm%20developer%20community/images/block%20diagrams/cortex-m%20processor/cortex-m33.svg>

External Interrupt/Event Controller (EXTI)

- EXTI connects GPIO pin changes to interrupt lines
- A pin can trigger on:
 - rising edge
 - falling edge
 - both edges
- In CubeMX, EXTI is configured from the pinout view
- The EXTI interrupt must also be enabled in the NVIC, this can be done via Configuration → NVIC Settings

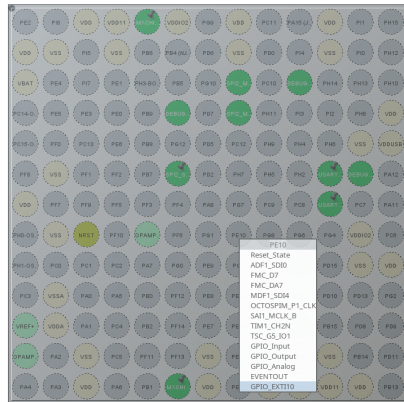
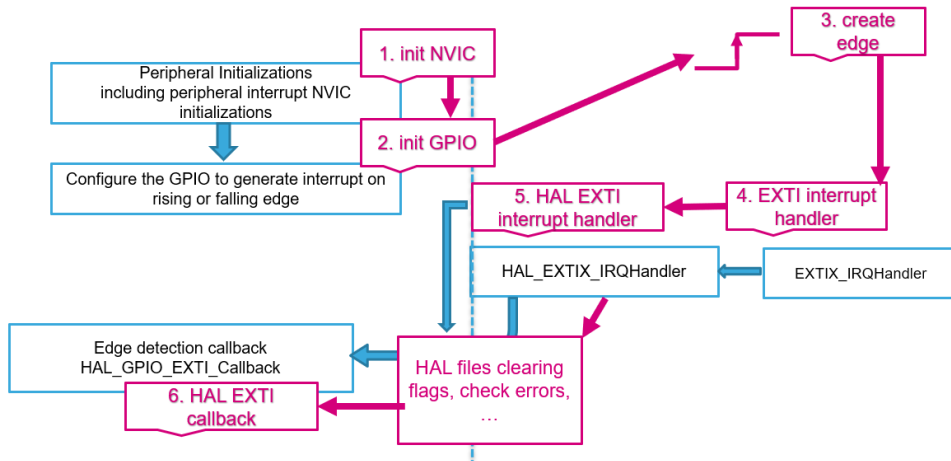


Figure: EXTI configuration of PE10 in the CubeMX pinout view

Interrupt path



Source: <https://wiki.st.com/stm32mcu/wiki/File:Flow.png>

Example: Non-blocking UART with HAL

- Start reception once with `HAL_UART_Receive_IT(...)`
- HAL calls `HAL_UART_RxCpltCallback(...)` when a byte arrives
- Reception in IT mode is one-shot
- Therefore, it must be restarted inside the callback

```
static uint8_t rx_byte;

HAL_UART_Receive_IT(&huart1, &rx_byte, 1);

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart == &huart1) {
        // process rx_byte
        HAL_UART_Receive_IT(&huart1, &rx_byte, 1);
    }
}
```

- Can detect motion, activity, inactivity, and wake-up events
- Supports configurable thresholds, duration settings, and output data rates
- Events can be routed to physical interrupt pins such as INT1
- On this board, INT1 is connected to MCU pin PE11
- PE11 must be configured as an EXTI input in CubeMX



Figure: ISM330DHCX motion sensor

Interrupt routing

Configuring PE11 as EXTI only prepares the MCU side. The sensor must also be configured to route a motion event to its INT1 output.

Tasks for today

1. Implement non-blocking UART CLI with interrupts
2. Read temperature and humidity using the BSP
3. Wake the MCU on motion using the IMU interrupt

What you should learn

The idea is for you to understand which layer is responsible for what: HAL, EXTI, NVIC, BSP and how they can work together. Additionally both interrupts and the BSP will be important for the tasks in the coming weeks!

Task 1: Non-Blocking UART Reception

In this task, you implement interrupt-based UART reception. Instead of blocking the CPU while waiting for input, the UART peripheral triggers an interrupt when a byte is received. This can for example be used to create a responsive UART based CLI.

- Configure USART1 in CubeMX
 - Use **Asynchronous**, 115200, 8N1
 - Enable the **USART global interrupt** in the NVIC
- Start interrupt-based reception
 - Use `HAL_UART_Receive_IT(...)`, what does it do?
 - Implement `HAL_UART_RxCpltCallback(...)`
 - Why must reception be restarted inside the callback?
- Pass information from callback to main loop
 - Store the received byte in `cmd`
 - Set a volatile flag such as `cmd_ready`
 - Why should this flag be volatile?
 - Handle the actual command in the main loop

Task 1: CLI Behaviour

- Send the help text once after initialization
- In the main loop continuously increment a counter to show that your program is not-blocking
- Use a `cmd_ready` flag to know when to process the command

```
switch (cmd) {  
  case 'c': // print current counter value  
    break;  
  case 'h': // print command list  
    break;  
  default: // print "Unknown"  
    break;  
}
```

Check

Typing `h` should print the command list. Typing `c` multiple times should print changing counter values.

Task 2: Sensor Setup and Data Acquisition

Use BSP to read temperature and humidity from the on-board HTS221 sensor and transmit the values over UART.

- Create a new CubeMX project.
 - Find out which I²C peripheral is connected to the environmental sensor.
 - Enable this I²C peripheral in CubeMX. This is good practice, although in most cases the BSP will handle this for you.
 - Enable USART1 in **Asynchronous** mode with 115200, 8N1 for debug output.
- Add the BSP files.
 - Find out where the STM32CubeU5 firmware package was installed on your device.
 - Copy the required BSP files into `Drivers/BSP/` of the newly generated project.
 - Which files belong to the board layer?
 - Which files belong to the HTS221 component driver?

Task 2: CMake and BSP Configuration

For your project to compile, you need to add the BSP source files and include paths to `CMakeLists.txt`.

- Add the required BSP source files with `target_sources(...)`.
 - Board sensor layer: `b_u585i_iot02a_env_sensors.c`
 - Board bus layer: `b_u585i_iot02a_bus.c`
 - Sensor component drivers dependencies
- Add the required include paths with `target_include_directories(...)`.
 - Board directory
 - `Components/Common`
 - `Components/<sensor>`
- Add the BSP configuration header.
 - Why does the BSP need `b_u585i_iot02a_conf.h`?
 - Create it from the BSP provided template if necessary.

Task 2: Read Sensor Values

After including all required files, the project should now compile. To start using the BSP, include `b_u585i_iot02a_env_sensors.h` in `main.c`. Your goal is to initialize the sensor and print the measured values in human-readable form over `USART1`.

- Initialize and enable the sensor.
 - Look into `BSP_ENV_SENSOR_Init(...)` and `BSP_ENV_SENSOR_Enable(...)`.
 - Which sensor instance is used on this board?
 - Which sensor functions do you want to enable?
- Read temperature and humidity values with `BSP_ENV_SENSOR_GetValue(...)`, then print them once per second in human-readable form over `USART1`.

Task 3: Wake-Up on Motion

Use the IMU to detect motion and wake the microcontroller through an external interrupt. The system sleeps when idle and reacts when the board is moved.

- Create a new CubeMX project.
- Configure debug output using USART1 in the usual configuration.
- Find out which MCU pin is connected to the IMU INT1.
 - Check the board schematic.
 - Find the ISM330DHCX INT1 pin and follow the signal to the STM32U5 pinout.
 - Configure the corresponding MCU pin as a GPIO EXTI input.
 - Enable the corresponding EXTI interrupt in the NVIC.

Important

The BSP can initialize and read the sensor, but it does not expose all interrupt-routing features required for this task.

Task 3: Motion Sensor Drivers and IMU Initialization

You will now use the BSP for generic sensor setup: initialization, enabling, ODR/full-scale configuration, and acceleration reads.

- Start by copying the required BSP files into your project and follow the same steps as in Task 2 to get the driver working.
- Include the required headers in `main.c`. This time, you also need to use the component driver directly.
 - `b_u585i_iot02a_motion_sensors.h`
 - `ism330dhcx.h`
- Initialize the accelerometer through the BSP, read up on the parameters used by the BSP.
 - Use motion sensor instance 0.
 - Enable `MOTION_ACCELERO`.
 - Set a moderate output data rate, e.g. 26 Hz.
 - Set a sensitive full-scale range, e.g. $\pm 2g$.

Task 3: Route the Sensor Interrupt

The BSP does not expose all advanced IMU interrupt features. Therefore, you will use the lower-level `ism330dxc*_` driver for interrupt routing.

- Configure the interrupt on the sensor side. Use the next slide for code hints.
 - Access `Motion_Sensor_CompObj[0]`.
 - Use `obj->Ctx` as the low-level driver context.
 - Configure activity/inactivity or wake-up detection.
 - Set the event threshold.
 - Route the selected event to INT1.
- Configure the interrupt on the MCU side.
 - INT1 is wired to the MCU pin found in the schematic.
 - Configure that pin as a GPIO EXTI input.
 - Enable the corresponding EXTI interrupt in the NVIC.

Task 3: Simplified Activity Interrupt Setup

```
static void configure_activity_interrupt(void)
{
    ism330dhcx_pin_int1_route_t int1_route = {0};

    ISM330DHCX_Object_t *obj =
        (ISM330DHCX_Object_t *)Motion_Sensor_CompObj[0];

    // Configure activity/inactivity event generation
    ism330dhcx_act_pin_notification_set(&obj->Ctx,
        ISM330DHCX_DRIVE_SLEEP_CHG_EVENT);

    // Enable activity logic, but do not let it change sensor power modes
    ism330dhcx_act_mode_set(&obj->Ctx,
        ISM330DHCX_XL_AND_GY_NOT_AFFECTED);

    // Set activity threshold
    ism330dhcx_wkup_threshold_set(&obj->Ctx, 2);

    // Route activity/inactivity event to INT1
    int1_route.md1_cfg.int1_sleep_change = 1;
    ism330dhcx_pin_int1_route_set(&obj->Ctx, &int1_route);
}
```

Task 3: EXTI Callback

- Implement the HAL EXTI callback for the configured edge
- The callback name depends on the HAL version and CubeMX configuration
 - rising edge: HAL_GPIO_EXTI_Rising_Callback(...)
 - falling edge: HAL_GPIO_EXTI_Falling_Callback(...)
 - older/common HAL style: HAL_GPIO_EXTI_Callback(...)
- Check the generated HAL driver or callback weak definitions if unsure

```
void HAL_GPIO_EXTI_Rising_Callback(uint16_t GPIO_Pin){  
    if (GPIO_Pin == <YOUR_GPIO_PIN>){  
        \\ DO work  
    }  
}
```

Rule

The callback should only record the event. Do not do expensive work inside the interrupt context.

Task 3: Sleep, Wake

Now all that is left to do is to implement the main loop and the EXTI callback. The program should enter sleep mode while no motion event is pending, wake up when the board is moved, and handle the event in the main loop.

- If no event is pending, one possible way to enter sleep mode is to suspend SysTick and then execute WFI. Why is suspending SysTick not always in your best interest? Why do we do it if we want sleep? You can read more on power management here: https://wiki.st.com/stm32mcu/wiki/Getting_started_with_PWR#Sleep_mode.

```
HAL_SuspendTick();  
HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);  
HAL_ResumeTick();
```

- Wake up on the IMU interrupt via the EXTI callback, read acceleration values, and print them over UART. Return to sleep after handling the event. Keep the callback short and handle the actual work in the main loop.

Questions?

Chair for Embedded Systems (CES)
Karlsruhe Institute of Technology (KIT)